

РАБОТА С ОБОРУДОВАНИЕМ

Нижний Новгород, 2025г.

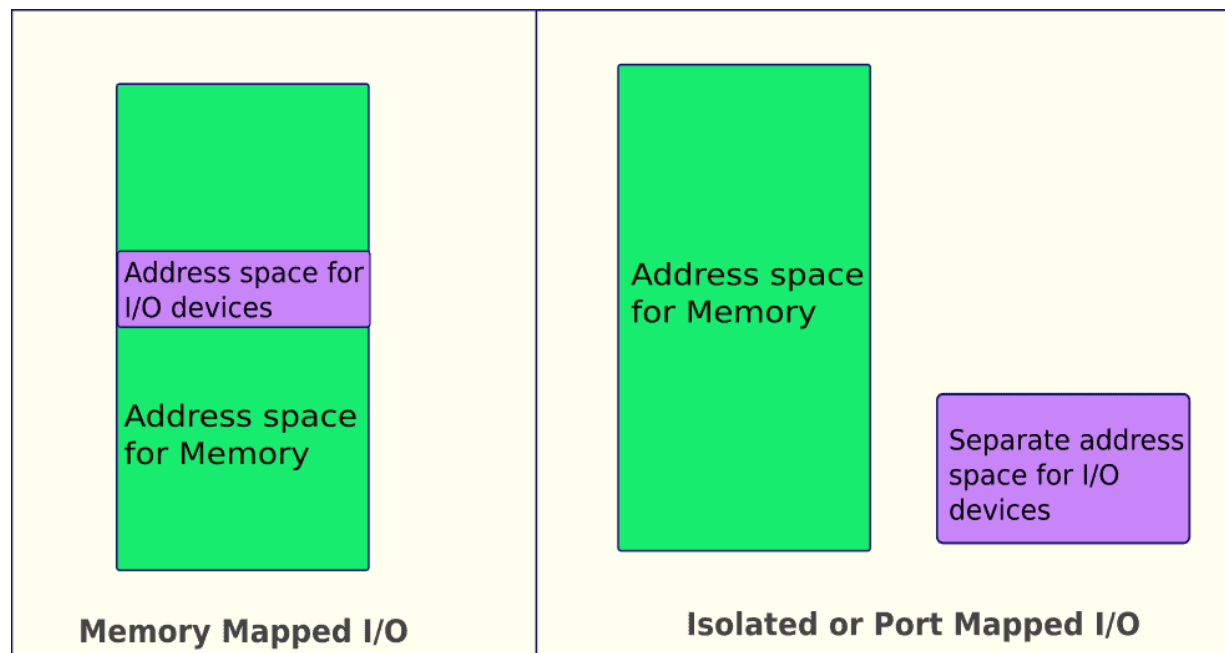
НИЗКОУРОВНЕВЫЙ ВВОД-ВЫВОД

Проблема: Процессор быстрый, периферийные устройства — медленные.

Решение: Специализированные механизмы взаимодействия

Два основных подхода:

- Port-Mapped I/O (Изолированный I/O)
- Memory-Mapped I/O (Отображаемый в память I/O)



РЕГИСТРЫ УСТРОЙСТВ

Что это: Небольшие, сверхбыстрые ячейки памяти *на самом устройстве*.

Назначение: Хранение состояния, управление, передача данных.

Типы регистров:

- Регистр данных (Data Register)
- Регистр состояния (Status Register)
- Регистр управления (Control Register)

PORT-MAPPED I/O (ИЗОЛИРОВАННЫЙ I/O)

Port-Mapped I/O (PMIO) — это метод взаимодействия CPU с периферийными устройствами, при котором для обмена данными используются **специальные порты ввода-вывода**, а не ячейки оперативной памяти.

- Существует отдельное адресное пространство для портов ввода-вывода.
- Адреса портов: Не совпадают с адресами оперативной памяти.
- Специальные инструкции: IN (прочитать из порта), OUT (записать в порт).

PORT-MAPPED I/O (ИЗОЛИРОВАННЫЙ I/O)

Достоинства	Недостатки
Не занимает адресное пространство RAM	Требует специальных инструкций (IN/OUT)
Простота декодирования для устройств	Менее гибкий, чем Memory-Mapped I/O
Четкое разделение доступа к памяти и устройствам	Сложнее для программирования

MEMORY-MAPPED I/O (ОТОБРАЖАЕМЫЙ I/O)

- Регистры устройств отображаются в общее адресное пространство оперативной памяти.
- Обращение: Обычные инструкции работы с памятью (MOV, LOAD, STORE).

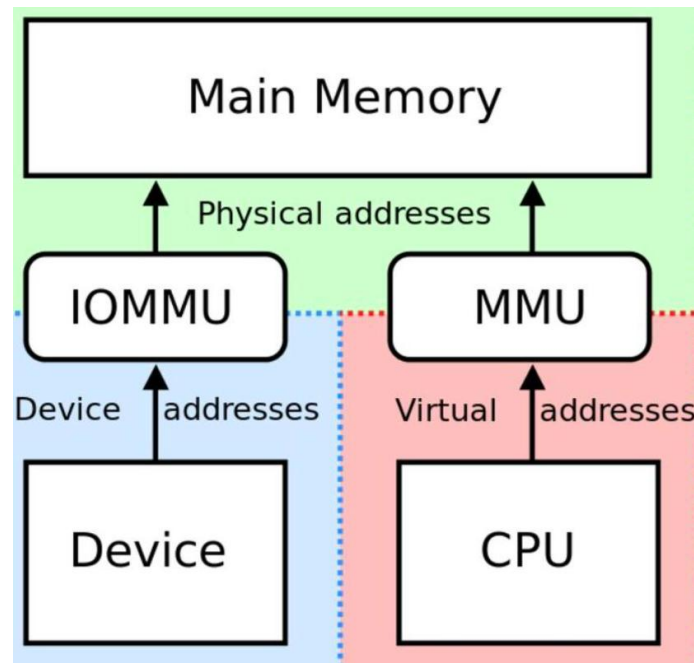
Плюсы: Упрощение программирования, единая модель доступа.

Минусы: "Съедает" часть адресного пространства, требует защиты со стороны MMU.

INPUT-OUTPUT MEMORY MANAGEMENT UNIT

MMU (Memory Management Unit) — это аппаратный блок в центральном процессоре (CPU), который управляет доступом к оперативной памяти.

IOMMU (Input-Output Memory Management Unit) — это аппаратный модуль, который управляет доступом периферийных устройств (таких как видеокарты, контроллеры дисков, сетевые адаптеры) к оперативной памяти компьютера



ЗАДАЧИ ММУ

- **Виртуальная память:** Каждая программа "видит" свое собственное изолированное адресное пространство (как будто у нее одна вся память), не зная о других процессах
- **Защита:** Программа не может получить доступ к памяти ядра ОС или других программ. Попытка приводит к ошибке (Segmentation Fault)
- **Пейджинг:** Позволяет использовать дисковое пространство в качестве расширения оперативной памяти (файл подкачки).

Ключевое отличие от ИОММУ: ММУ работает для **процессора и программ**, а ИОММУ — для **периферийных устройств** (видеокарт, сетевых карт и т.д.).

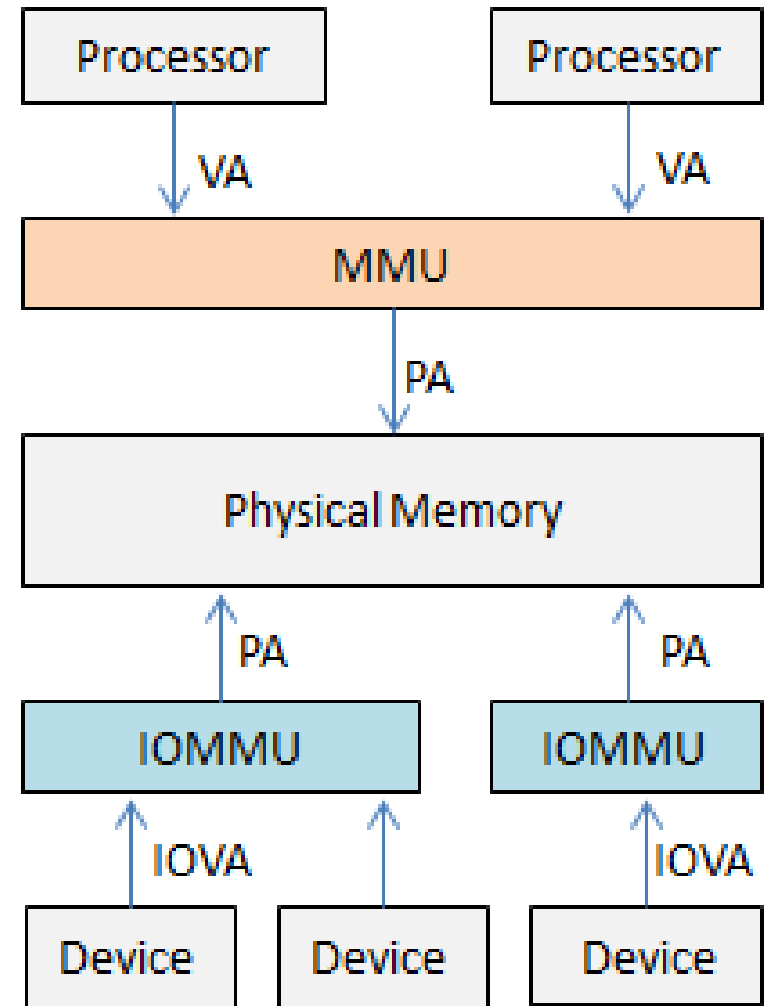
ЗАДАЧИ ИОММУ

- **Безопасность:** Устройству доступна только разрешенная область памяти (защита от DMA-атак)
- **Виртуализация:** Позволяет напрямую назначать физическое устройство виртуальной машине
- **Совместимость:** Старые 32-битные устройства могут работать с памятью за пределами 4 ГБ.

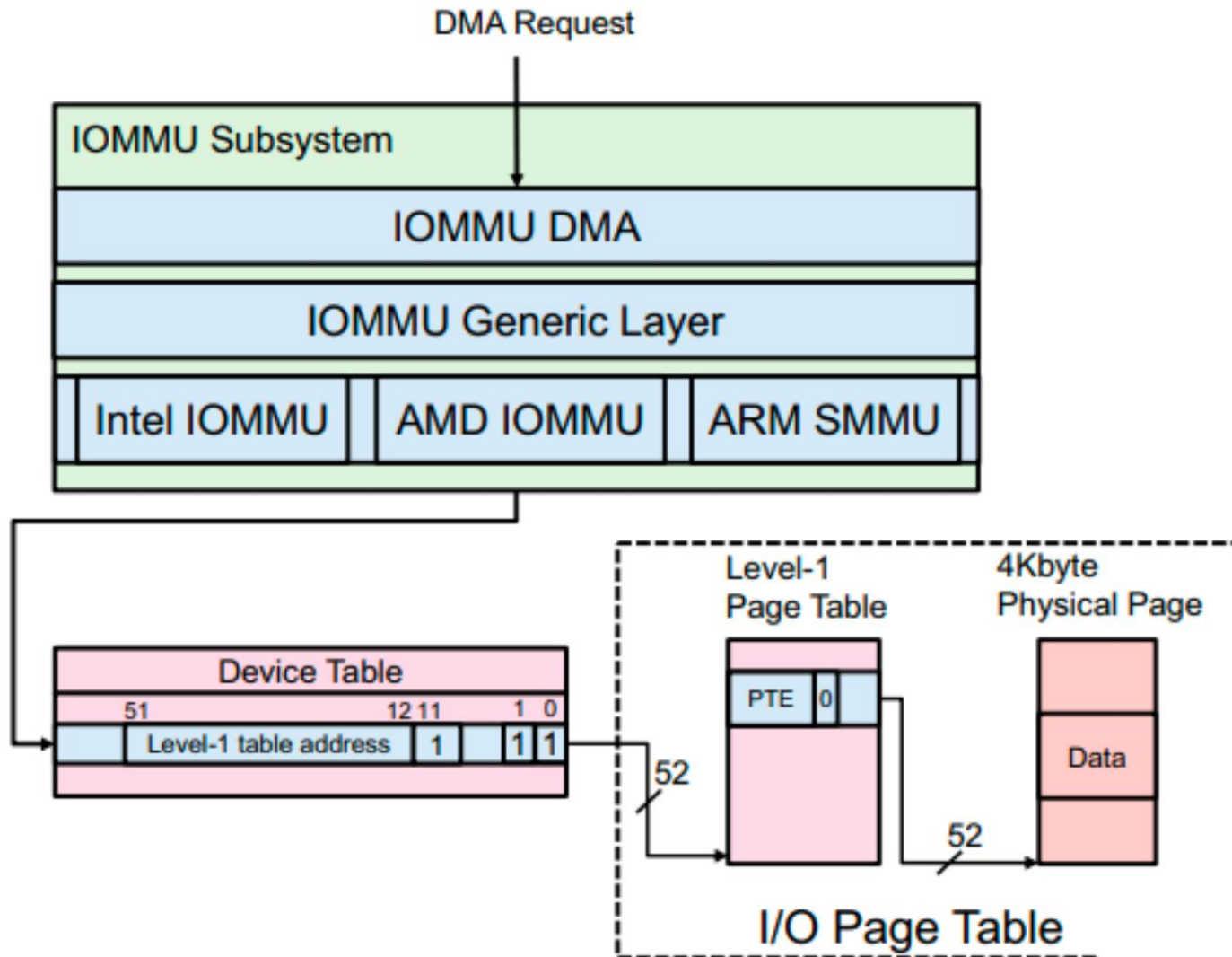
Простая аналогия: ИОММУ — это **пропускной пункт** для устройств, который проверяет, куда именно в памяти им разрешено обращаться.

ЗАДАЧИ ИОММУ

1. **Драйвер устройства** в ОС настраивает ИОММУ, создавая для устройства таблицу трансляции адресов
2. Когда устройству нужно выполнить DMA-запрос, оно использует **IO Virtual Address (IOVA)**
3. **ИОММУ перехватывает** этот запрос
4. ИОММУ ищет в своей таблице, какому **Physical Address (PA)** соответствует данный IOVA
5. ИОММУ перенаправляет запрос с правильным физическим адресом к контроллеру памяти



ПРЕОБРАЗОВАНИЕ АДРЕСОВ IOMMU



MEMORY-MAPPED I/O (ОТОБРАЖАЕМЫЙ I/O)

Критерий	Port-Mapped I/O	Memory-Mapped I/O
Инструкции	Специальные (IN/OUT)	Обычные (работы с памятью)
Адр. пространство	Отдельное	Единое с памятью
Простота	Сложнее	Проще
Гибкость	Меньше	Больше
Безопасность	Выше	Требует MMU

!!! В современных системах часто используется гибридный подход

ПРИМЕР: ЧТЕНИЕ СИМВОЛА С КЛАВИАТУРЫ (PIO)

Алгоритм:

- Читаем **Регистр состояния** клавиатуры.
- Проверяем бит "Готовность данных" (Data Ready).
- Если бит установлен -> читаем **Регистр данных** и получаем символ.
- Если нет -> возвращаемся к шагу 1.

Проблема: Процессор постоянно занят опросом, тратит время впустую.

ПРОБЛЕМА МЕТОДА ОПРОСА (POLLING)

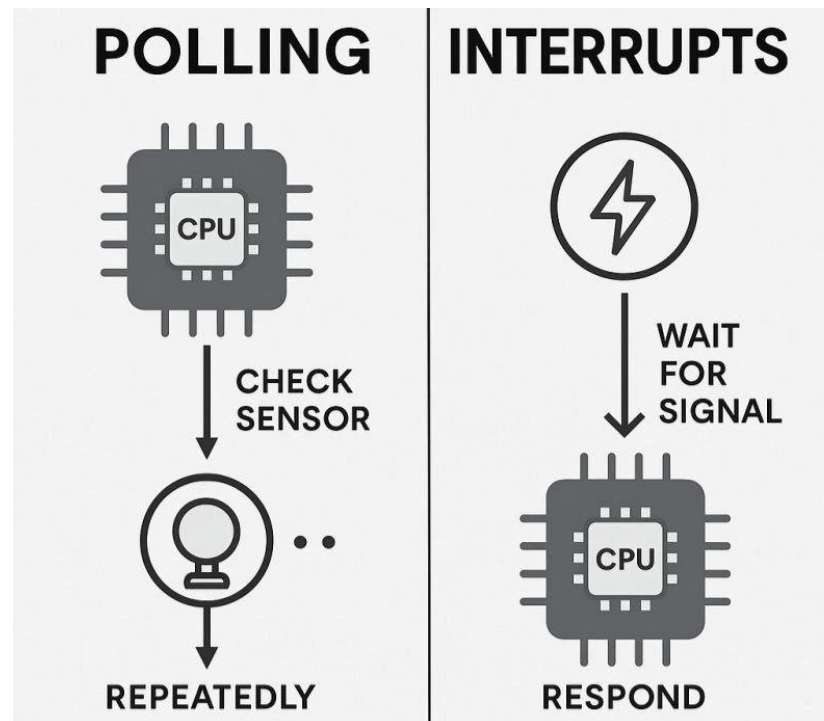
- Холостая работа процессора
- Низкая отзывчивость системы
- Плохая масштабируемость (десятки устройств?)

Что делать, если устройство работает очень медленно (например, клавиатура)?

ПРЕРЫВАНИЯ И ИХ ОБРАБОТКА

Суть: Устройство само уведомляет процессор, когда оно готово к обмену

Преимущество: Процессор свободен для выполнения других задач, пока устройство не подаст сигнал



ПРЕРЫВАНИЯ (IRQ)

IRQ (Interrupt ReQuest) — это сигнал, который устройство отправляет процессору, чтобы сообщить о том, что оно требует его внимания.



АППАРАТНЫЕ ПРЕРЫВАНИЯ (IRQ)

Как работает:

- **Устройству нужен квант времени** (например, клавиша нажата, данные получены по сети, диск прочитал сектор)
- **Устройство отправляет сигнал IRQ** на контроллер прерываний
- **Процессор прерывает текущую задачу** и выполняет специальную процедуру — обработчик прерывания (ISR)
- **Обработчик обслуживает устройство** (забирает данные, подтверждает операцию)
- **Процессор возвращается** к прерванной задаче

МАСКИРОВАННЫЕ ПРЕРЫВАНИЯ

Маскируемые и немаскируемые прерывания — это два типа аппаратных прерываний, которые отличаются возможностью их блокировки процессором.

Характеристика	Маскируемые	Немаскируемые (NMI)
Возможность блокировки	Да	Нет
Важность	Обычные события	Критические ошибки
Источники	Устройства ввода-вывода	Критические аппаратные сбои
Обработка	Может быть отложена	Немедленная

ПРОГРАММНЫЕ ПРЕРЫВАНИЯ

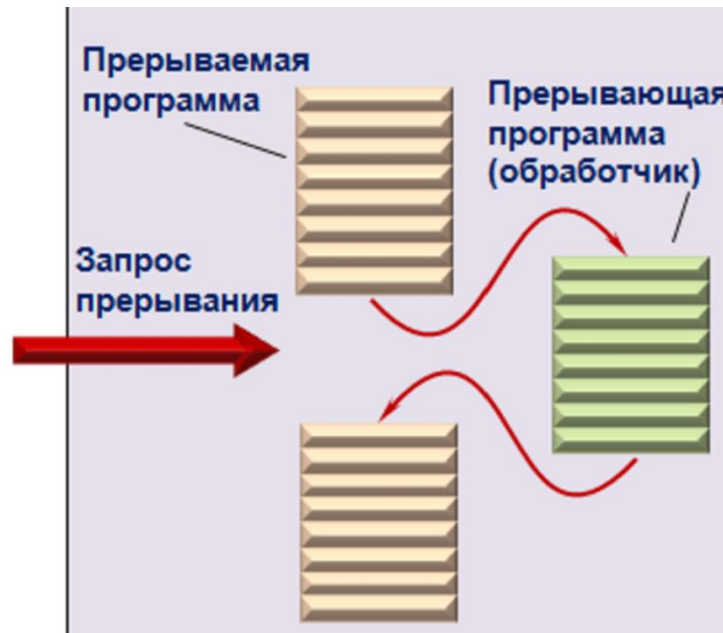
- **Программные прерывания:** Вызываются специальной инструкцией (INT в x86). Используются для системных вызовов
- **Исключения (Exceptions):** Возникают при особых условиях работы процессора (деление на ноль, доступ к невалидной памяти, page fault)



МЕХАНИЗМ ОБРАБОТКИ ПРЕРЫВАНИЯ (1)

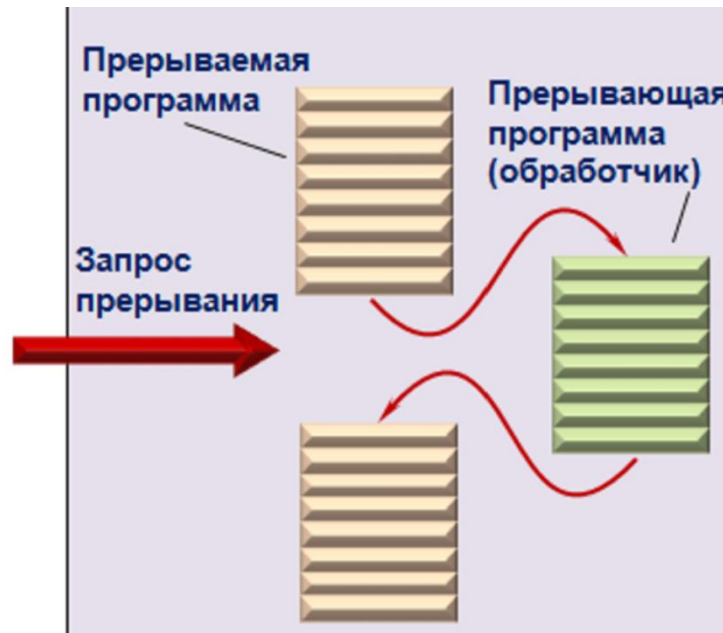
Порядок действий процессора при возникновении прерывания:

1. **Завершение текущей инструкции**
2. **Определение номера прерывания**
3. **Сохранение контекста:** Программный счетчик (IP), регистры флагов (EFLAGS) и др. помещаются в стек



МЕХАНИЗМ ОБРАБОТКИ ПРЕРЫВАНИЯ (1)

3. **Поиск обработчика:** Используется таблица векторов прерываний (Interrupt Vector Table, IVT) или таблица дескрипторов прерываний (IDT)
4. **Переход:** Загрузка адреса обработчика в программный счетчик
5. **Выполнение кода обработчика** прерывания (ISR)



ОБРАБОТЧИК ПРЕРЫВАНИЯ (ISR)

ISR (Interrupt Service Routine) – это блок кода, ассоциированный с конкретным прерыванием

Что делает ISR:

- Сохраняет остальные регистры
- Определяет, какое устройство вызвало прерывание.
- Обслуживает устройство (читает/пишет данные)
- Посылает сигнал конца прерывания (End of Interrupt, EOI) контроллеру
- Восстанавливает регистры
- Выполняет инструкцию возврата из прерывания (IRET)

ПРИОРИТЕТЫ И МАСКИРОВАНИЕ ПРЕРЫВАНИЙ

Приоритеты: Определяют, какое прерывание будет обработано первым

Маскирование: Возможность временно запретить (замаскировать) прерывания определенного уровня или все (cli/sti)

NMI (Non-Maskable Interrupt): Немаскируемое прерывание для критических ошибок (например, аппаратный сбой).

ПРЕРЫВАНИЯ VS. ОПРОС: ИТОГИ

Характеристика	Опрос (Polling)	Прерывания (Interrupts)
Загрузка CPU	Высокая	Низкая
Время отклика	Непредсказуемое	Малое, предсказуемое
Сложность	Низкая	Высокая (аппаратная и программная)
Использование	Простые системы, критические секции	Подавляющее большинство современных систем

DMA И БУФЕРИЗАЦИЯ

ПРОБЛЕМА ПЕРЕНОСА ДАННЫХ

Сценарий: Передача большого блока данных с диска в память (PIO mode)

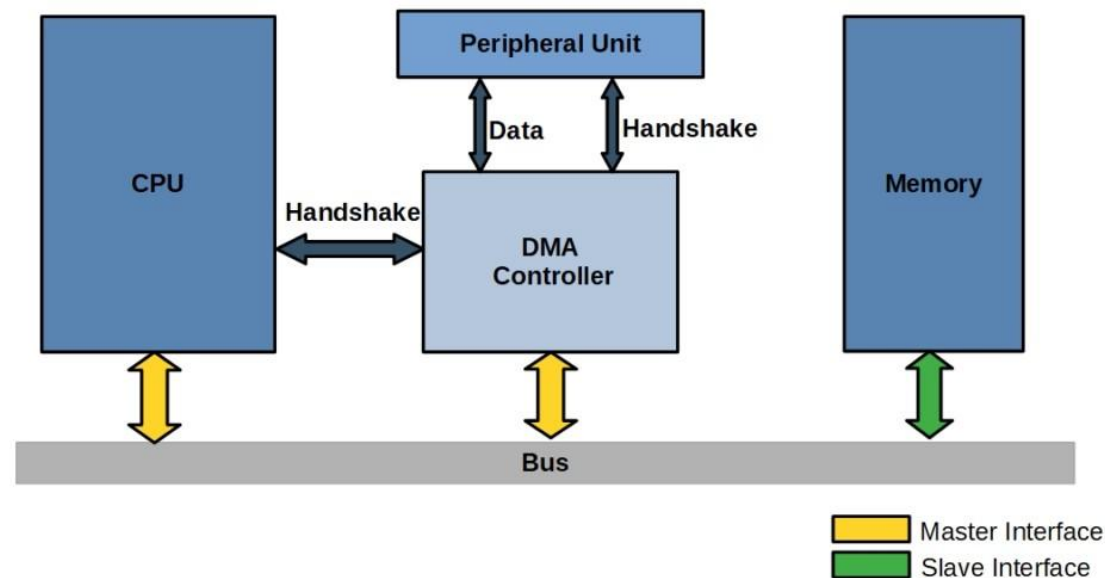
Проблема: Каждое слово данных должно быть прочитано процессором из порта диска и записано в память

Результат: Процессор "заблокирован" на время всей передачи

РЕШЕНИЕ — ПРЯМОЙ ДОСТУП К ПАМЯТИ (DMA)

DMA (Direct Memory Access) - специализированный контроллер, который берет на себя пересылку данных между устройствами и памятью.

Роль процессора: Инициализировать DMA-контроллер, а затем заниматься другими задачами.



КАК РАБОТАЕТ DMA (1)

1. Запрос: Процессор настраивает DMA-контроллер:

- Указывает **источник** данных (адрес устройства).
- Указывает **назначение** (адрес в памяти).
- Указывает **размер** блока данных (количество байт).
- Запускает передачу.

2. Передача: DMA-контроллер самостоятельно управляет шиной:

- Запрашивает доступ к шине (Bus Request).
- Получает разрешение (Bus Grant) от процессора.
- Читает данные из устройства и записывает в память.
- Отслеживает количество переданных байт.

КАК РАБОТАЕТ DMA (2)

3. **Завершение:** После передачи всего блока:

- DMA-контроллер освобождает шину
- Посылает процессору **прерывание**, сигнализируя об окончании передачи

4. **Обработка:** Процессор, получив прерывание, узнает, что данные ГОТОВЫ